

DFCMS — Architektura systemu i workflow

Dokument wygenerowany z repozytorium dfopscms · 2026-06-03 · Źródła: ARCHITECTURE.md, WORKFLOW.md, PROJECT_STATE.md

Spis treści

- 1. Podział logiczny systemu
- 2. Środowiska Staging / Production
- 3. Przepływ danych (architektura)
- 4. Kluczowe ścieżki biznesowe
- 5. Edge Functions
- 6. Lokalny development
- 7. Baza danych i Supabase CLI
- 8. Stripe (test vs live)
- 9. Wdrożenie (deploy)
- 10. Gałęzie Git i bezpieczeństwo

1. Podział logiczny systemu

Warstwa	Odpowiedzialność	Technologie
Frontend	Landing, szablony, panel CMS, routing wielodomenowy	HTML, JS, Alpine.js, Tailwind CDN, config.js
Hosting	CDN, preview, custom hostnames (SaaS)	Cloudflare Pages, _middleware.js
Backend	Auth, treść, rozliczenia, obrazy	Supabase: PostgreSQL, Auth, Storage, RLS
Serverless	Płatności, domeny, cron, opinie, alerty	Supabase Edge Functions (Deno)
Płatności	Checkout, Portal, webhooks	Stripe (Test / Live)
DNS klientów	Własne domeny	Cloudflare for SaaS + add-custom-domain
Observability	Błędy, alerty ops	Sentry, telegram-webhook

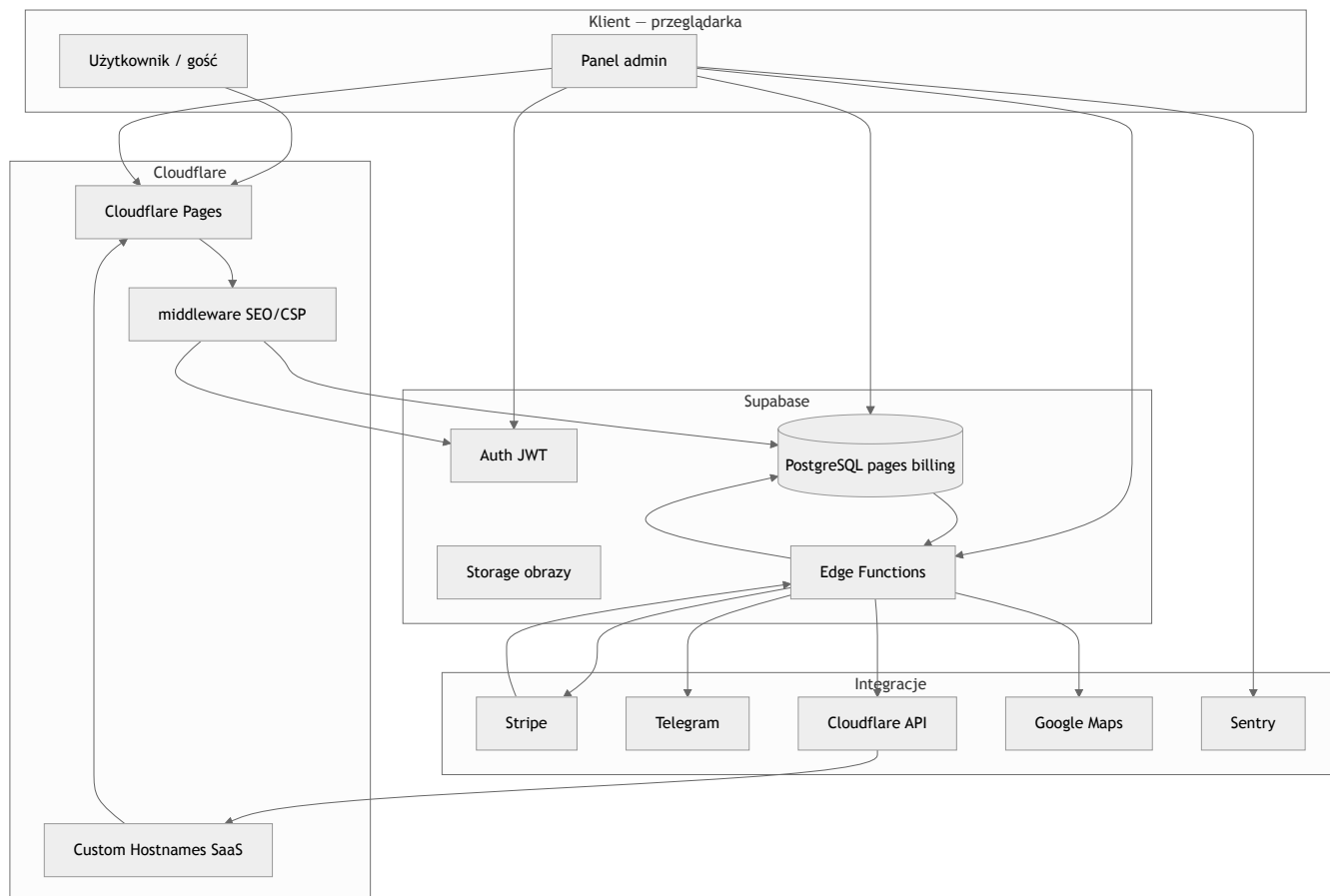
2. Środowiska Staging / Production

Środowisko	Git	Frontend	Supabase project-ref	Stripe
Staging	staging	staging.dfcms.pl, *.pages.dev	asxrdsprrbvjvgcckh	Test mode

Production	main	dfcms.pl, {slug}.dfcms.pl	tawywecinkubmouyprab	Live mode
-------------------	------	---------------------------	----------------------	-----------

config.js (bez Dockera): hostname `localhost`, `staging.dfcms.pl`, `*.pages.dev` → Staging API; pozostałe hosty → Production.

3. Przepływ danych



4. Kluczowe ścieżki biznesowe

- Rejestracja / edycja** — Auth + PostgREST (`pages`, `draft_content` / `content`), RLS + anon key.
- Publikacja** — `draft_content` → `content` ; publicznie tylko `content` ; podgląd: ? `dfcms_preview=1` .
- Płatność** — create-checkout → Stripe → stripe-webhook / sync → billing_profiles + pages.billing_plan.
- Własna domena** — add-custom-domain → Cloudflare Custom Hostname → pages.custom_domain.
- Alerty** — Sentry / Database Webhooks → telegram-webhook → Telegram.

5. Edge Functions

Funkcja	Rola
---------	------

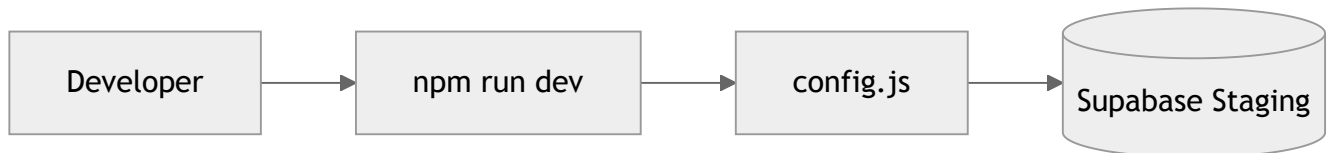
create-checkout	Stripe Checkout (plan, interval)
create-portal-session	Stripe Customer Portal
stripe-webhook	Zdarzenia Stripe → DB
sync-stripe-subscription	Ręczna synchronizacja subskrypcji
add-custom-domain	Cloudflare Custom Hostname
get-google-reviews	Google Places / opinie
expire-trial-pages	Cron trial + purge
telegram-webhook	Router alertów → Telegram

Współdzielona logika Stripe: `supabase/functions/_shared/stripeBilling.ts`

6. Lokalny development

```
npm install
npm run dev # http://localhost:3000 → Supabase Staging (bez supabase start / Docker)
```

Wymagania Staging: redirect URL `http://localhost:3000/admin.html`, Stripe Test w Secrets.



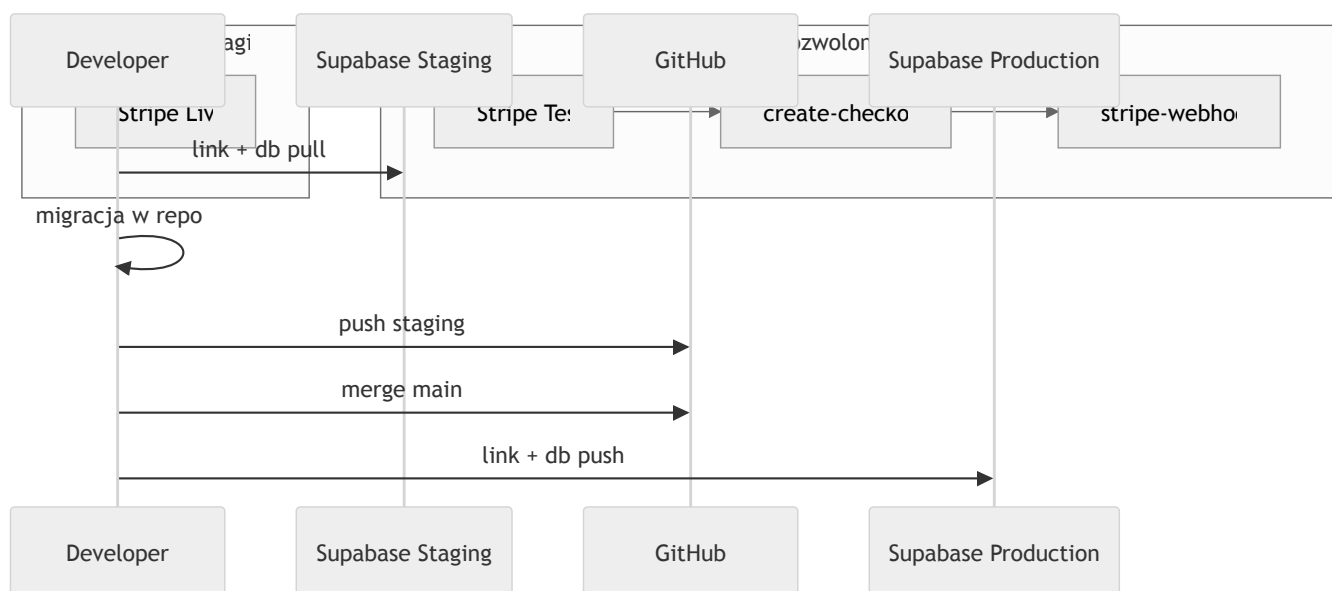
7. Baza danych i Supabase CLI

Przełączanie project-ref (jeden link na raz)

Cel	project-ref	Git front	Komenda
Staging	asxrdsprrbvjvgcckh	staging	npm run supabase:link:staging
Production	tawywecinkubmouyprab	main	npm run supabase:link:production

Sprawdzenie: `npm run supabase:linked`

Uwaga: `db push` stosuje migracje SQL z repo na zlinkowany projekt — nie kopiuje danych ze Stagingu na Prod.



Typowy cykl migracji

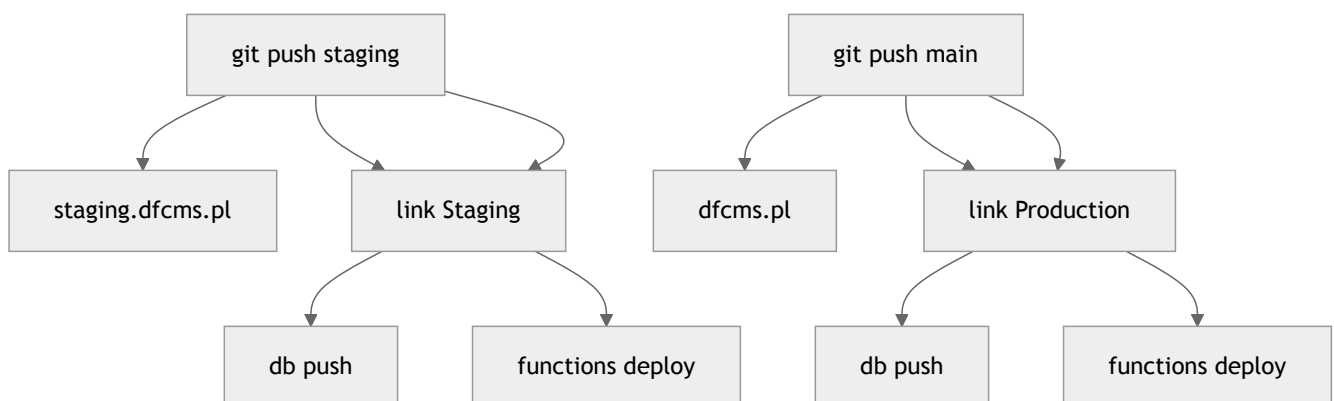
```
supabase link --project-ref asxrdsprrbvjvgcckh
supabase db pull
```

```
# edycja supabase/migrations/*.sql
git push origin staging
# po QA:
supabase link --project-ref tawywecinkubmouyprab
supabase db push
```

8. Stripe — test vs live

- Staging / localhost: wyłącznie Stripe **Test** i webhook na projekt Staging.
- Production: Live keys, Live Price ID, osobny webhook.
- Karty testowe: 4242 4242 4242 4242 — nigdy live na Stagingu.

9. Wdrożenie (deploy)



Skróty npm

```
npm run deploy:db:staging
npm run deploy:functions:staging
git push origin staging

npm run deploy:db:production
npm run deploy:functions:production
git push origin main
```

Checklist przed Production

- Migracje przetestowane na Staging
- Edge Functions + Secrets Live na Production

- Cloudflare Pages (main) → prod Supabase URL/anon
- Stripe webhook Live → prod stripe-webhook
- Database Webhooks → telegram-webhook (opcjonalnie)

10. Gałęzie Git i bezpieczeństwo

Gałąź	Cel
staging	QA, Stripe Test, Supabase Staging
main	Produkcja — Cloudflare + Supabase Production

Service role, STRIPE_SECRET_KEY, tokeny CF/Telegram — tylko Supabase Secrets / CI, nigdy w Git.

Repozytorium: dfopscms · Aktualizuj ARCHITECTURE.md i WORKFLOW.md przy zmianach infrastruktury.